Perceptron e MLP

Resumo da Aula 6.12

Objetivo: entender como um neurônio artificial faz previsões e como uma rede com camada oculta aprende, ajustando pesos e biases por backpropagation e descida do gradiente.

1. Onde isso entra no machine learning?

Na aula, fazemos **classificação supervisionada**: recebemos características **X** e respostas conhecidas **y**. O modelo calcula previsões, compara com essas respostas e ajusta seus parâmetros durante o treinamento.

O **Perceptron** usa um único neurônio. O **MLP**, ou perceptron multicamadas, organiza vários neurônios em camadas e pode representar relações mais complexas.

2. O neurônio: entradas, pesos, bias e ativação

O neurônio multiplica cada entrada pelo peso correspondente, soma um bias e aplica uma função de ativação:

$$z = w_1 x_1 + w_2 x_2 + b, \qquad a = f(z)$$

- **Entradas** x**:** características do exemplo.
- **Pesos** w**:** controlam como cada entrada participa da soma.
- **Bias** b**:** ajuste somado à conta; ajuda a deslocar o ponto de decisão.
- **Ativação** f**:** transforma a soma z na saída do neurônio.

Exemplo didático: para um aluno com 3 horas de estudo e frequência 0,8, escolhemos pesos 0,5 e 1, e bias -2:

$$z = (0{,}5 \times 3) + (1 \times 0{,}8) - 2 = 0{,}3$$

Os valores foram escolhidos para ilustrar a conta. No treinamento, pesos e bias são ajustados usando os exemplos conhecidos.

3. Como o Perceptron aprende

A ativação degrau pode usar classes 0 e 1: prevê 1 se $z \geq 0$ e 0 se $z < 0$. Na regra de treinamento dos slides, as classes são -1 **e** $+1$, com a mesma divisão em $z = 0$.

Quando erra, o Perceptron atualiza:

$$w \leftarrow w + \eta\, y\, x, \qquad b \leftarrow b + \eta\, y$$

y é a resposta correta (-1 ou $+1$), e $\eta > 0$ é a **taxa de aprendizado**, que controla o tamanho do ajuste. Uma **época** é uma passagem por todos os exemplos. Podemos parar em uma época sem erros ou ao atingir o limite definido.

Fonte: Aula 6.12 - Perceptron e MLP com Backpropagation. Exemplos adaptados.

4. A limitação linear e o XOR

Com duas entradas, o Perceptron separa as classes por uma **reta**. Ele consegue acertar todos os exemplos apenas quando existe uma separação linear adequada. O XOR mostra uma situação em que isso é impossível:

x_1	x_2	**XOR**
0	0	0
0	1	1
1	0	1
1	1	0

Entradas iguais dão 0; diferentes dão 1. No gráfico, os dois pontos de cada classe ocupam cantos opostos. **Nenhuma reta separa as duas classes perfeitamente.** Mais épocas não eliminam essa limitação do modelo.

5. O MLP e a camada oculta

A rede da aula tem estrutura $2 \to 4 \to 1$:

- **Entrada:** os dois valores do XOR, x_1 e x_2.
- **Camada oculta:** quatro neurônios, cada um com seus próprios pesos e bias.
- **Saída:** um neurônio que combina as quatro ativações ocultas.

A camada oculta cria uma **nova representação dos dados**. Com uma ativação não linear, essa transformação permite ao MLP representar o XOR. Na rede $2 \to 4 \to 1$, as camadas com parâmetros são a oculta e a de saída.

6. Sigmoid e propagação para a frente

Nesse MLP, usamos a ativação **sigmoid**, uma curva suave com saída entre 0 e 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

Por exemplo, $\sigma(-2) \approx 0{,}119$, $\sigma(0) = 0{,}5$ e $\sigma(2) \approx 0{,}881$. Na camada oculta, guardamos esses valores contínuos. Na saída final, prevemos **1 se** $a \geq 0{,}5$ e **0 se** $a < 0{,}5$.

O **forward propagation** calcula a previsão com os parâmetros atuais:

$$\begin{aligned} Z_1 &= XW_1 + b_1, \quad A_1 = \sigma(Z_1), \\ Z_2 &= A_1W_2 + b_2, \quad A_2 = \sigma(Z_2). \end{aligned}$$

Z representa somas antes da ativação; A, os valores depois dela. Cada linha de X é uma amostra. Para m amostras, A_1 tem formato $m \times 4$ e A_2, $m \times 1$.

7. A perda mede o erro da previsão

A **binary cross-entropy (BCE)** compara a saída contínua a com a resposta correta y, codificada como 0 ou 1. Para uma amostra:

$$L = -\Big[y \ln(a) + (1 - y) \ln(1 - a)\Big]$$

Se $y = 1$, a conta fica $-\ln(a)$; se $y = 0$, fica $-\ln(1 - a)$. A perda do conjunto é a **média das perdas das amostras**.

Resposta correta	Saída da rede	Perda aproximada
1	0,2	1,609
1	0,8	0,223
1	0,99	0,010

Duas previsões podem acertar a classe e ter perdas diferentes. Treinamos usando a saída contínua, preservando a informação de quão próxima ela está da resposta correta.

8. Backpropagation e regra da cadeia

O **backpropagation** calcula como pequenas mudanças em cada parâmetro afetam a perda. Ele percorre a rede da saída para a camada oculta, usando a **regra da cadeia**:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}$$

O peso altera a soma, que altera a ativação, que altera a perda. O resultado é um **gradiente** para cada peso e bias. Um gradiente positivo indica que aumentar um pouco o parâmetro tende a aumentar a perda; um negativo indica a tendência contrária.

9. Exemplo: o erro voltando pela rede

Para **sigmoid + BCE**, em uma única amostra, o gradiente na soma de saída simplifica para $g_2 = a_2 - y$. Se a saída é 0,8 e a resposta correta é 1:

$$g_2 = 0{,}8 - 1 = -0{,}2$$

Se uma ativação oculta vale 0,6, o gradiente do peso que a liga à saída é $-0{,}2 \times 0{,}6 = -0{,}12$. O gradiente do bias de saída é $-0{,}2$.

Para voltar ao neurônio oculto, suponha que o peso dessa conexão valha 0,8. Multiplicamos pelo peso e pela derivada da sigmoid oculta:

$$g_1 = (-0{,}2) \times 0{,}8 \times 0{,}6(1 - 0{,}6) = -0{,}0384$$

O gradiente de cada peso de entrada desse neurônio é g_1 vezes a entrada correspondente. Para o bias oculto, usamos g_1. Com várias amostras, os gradientes consideram a perda média.

10. Atualizar os parâmetros e repetir

O backpropagation calcula os gradientes. A **descida do gradiente** usa esses valores para atualizar os parâmetros no sentido oposto ao gradiente:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

Se um peso é 0,8, seu gradiente é $-0{,}12$ e a taxa é 0,1:

$$w_{\text{novo}} = 0{,}8 - \Big(0{,}1 \times (-0{,}12)\Big) = 0{,}812$$

Gradiente negativo faz o parâmetro aumentar; positivo faz diminuir, para uma taxa positiva. Buscamos reduzir a perda em passos sucessivos.

O ciclo de treinamento do MLP é:

1. **Forward:** calcular as previsões.
2. **Perda:** comparar as saídas contínuas com as respostas.
3. **Backward:** calcular todos os gradientes.
4. **Atualização:** ajustar os pesos e biases.

Todos os gradientes são calculados com os parâmetros daquele forward. Só depois fazemos as atualizações. No guiado, usamos os quatro casos do XOR juntos, com uma atualização por época: treinamento *full batch*.

11. Cuidados para o treinamento funcionar

- **Inicialização:** pesos aleatórios diferenciam os neurônios. Se todos começarem com pesos e biases iguais, podem aprender a mesma coisa. Os biases podem começar em zero.
- **Semente:** permite repetir a mesma inicialização, mantendo os dados e as configurações.
- **Taxa de aprendizado:** muito pequena pode tornar o treino lento; muito grande pode provocar oscilações ou aumento da perda.
- **Formatos:** na implementação em lote, mantenha y e A_2 com o mesmo formato $m \times 1$. Cada gradiente deve ter o formato de seu parâmetro.
- **Estabilidade:** no código da BCE, limitamos numericamente as saídas para evitar calcular $\ln(0)$.

12. Como verificar o aprendizado

A **curva de perda** mostra a evolução do erro médio. Conferimos também as **classes previstas**, aplicando o limite de 0,5 às saídas. Na ordem $(0,0)$, $(0,1)$, $(1,0)$ e $(1,1)$, o resultado esperado é **0, 1, 1, 0**.

No XOR, verificamos a tabela lógica completa. Em problemas reais de ML, avaliamos também dados separados do treino para medir a **generalização**. Acertar exemplos usados no treinamento, sozinho, não demonstra desempenho em dados novos.